MarkerPilot: A Self-Driving Robot Car Guided by a Ceiling Camera and Reinforcement Learning

Overhead ArUco-marker localisation, Kalman filtering, Q-learning and ESP-NOW radio control

Category	Level	Main board
Robotics	High school	ESP32 + ESP32-C3

1. ABSTRACT

MarkerPilot is a small robot car that drives itself to any point I click on a laptop screen, or to a printed goal marker placed on the floor. A USB camera on the ceiling watches the floor. A printed ArUco marker on top of the car tells the computer exactly where the car is and which way it faces, 30 times every second. A Kalman filter cleans up the camera measurements and predicts the car's position a little into the future to cancel radio and camera delay. The driving decision is made either by a classic controller or by a reinforcement-learning agent that learned to drive by practising thousands of times in a simulator. Commands travel from the laptop to an ESP32-C3 USB stick and then over the ESP-NOW radio to the ESP32 on the car. Three ultrasonic sensors on the car let it slow down, steer around and back away from obstacles, and the car itself refuses to drive forward into something closer than a safe distance even if the radio link drops a command. In 300 randomised simulator tests the system reached 100% of goals on an open floor and 91% with obstacles in the way.

2. AIM OF THE PROJECT

- Build a two-wheel robot car controlled by an ESP32 over the low-delay ESP-NOW radio.
- Track the car's position and heading using only a ceiling camera and a printed ArUco marker.
- Make the car drive smoothly to clicked waypoints or to a goal marker, and stop accurately.
- Teach a reinforcement-learning (RL) agent to drive, and compare it fairly with a classic controller.
- Build a realistic simulator so everything can be tested without the real car.
- Make a desktop app that runs on Windows, Linux and macOS and finds every connected camera.

3. PROBLEM STATEMENT AND NEED

Warehouse and hospital robots usually carry expensive sensors (LiDAR, wheel encoders, cameras) to know where they are, and wheel slip makes their position estimate drift. A single cheap camera on the ceiling can see the whole floor at once with no drift at all. The robot itself then only needs motors, a cheap microcontroller and simple ultrasonic sensors. This makes the robot light, cheap and easy for schools and small workshops to build.

4. COMPONENTS USED AND COST

Component	Model	Qty	Purpose	Approx. cost
Car controller	ESP32-WROOM-32 dev board	1	Receives radio commands, drives motors, reads sensors	Rs. 420
USB radio gateway	ESP32-C3 SuperMini	1	Plugs into the laptop, relays commands over ESP-NOW	Rs. 300
Motor driver	L298N dual H-bridge	1	Speed and direction of both motors	Rs. 180
Motors + wheels	BO geared DC motors	2	Left and right drive wheels	Rs. 160
Chassis	Acrylic base + caster	1	Holds the parts	Rs. 280
Ultrasonic sensors	HC-SR04	3	Left (40°), front and right (40°) obstacle sensing	Rs. 270
Camera	1080p USB webcam	1	Mounted on the ceiling looking down	Rs. 1,200

Component	Model	Qty	Purpose	Approx. cost
Battery	2S 7.4 V Li-ion pack	1	Powers motors and the ESP32	Rs. 360
Buzzer	Active buzzer	1	Obstacle warning beep	Rs. 20
Markers	Printed ArUco 4x4 (IDs 0 and 1)	2	Car marker and goal marker	Rs. 10
Wires, resistors	1 kΩ + 2 kΩ per echo line; 100 kΩ + 33 kΩ	1 set	Echo dividers, battery-voltage divider	Rs. 60
Total				Rs. 3,260

Table 1: Bill of materials (approximate local prices)

5. CIRCUIT CONNECTIONS

The ultrasonic ECHO pins give a 5 V pulse but ESP32 pins accept only 3.3 V, so each ECHO line goes through a 1 kΩ / 2 kΩ voltage divider. The L298N enable jumpers stay on, so the four IN pins carry PWM directly.

Part pin	ESP32 pin	What it does
L298N IN1 / IN2	GPIO 27 / 26	Left motor forward / backward (PWM)
L298N IN3 / IN4	GPIO 25 / 33	Right motor forward / backward (PWM)
Left sonar TRIG / ECHO	GPIO 5 / 18	Left 40° obstacle distance
Front sonar TRIG / ECHO	GPIO 22 / 23	Front obstacle distance
Right sonar TRIG / ECHO	GPIO 19 / 21	Right 40° obstacle distance
Buzzer	GPIO 4	Beep on command from the app
Battery sense (optional)	GPIO 34	Battery + through 100 kΩ / 33 kΩ divider
Status LED	GPIO 2 (on board)	Fast flash = searching, slow flash = linked
Gateway (ESP32-C3)	USB to laptop	Serial 115200: 'D,l,r' drive, 'G,cm' guard, 'B' beep; replies 'T,...' and 'S,...'

Table 2: Wiring of the car and gateway

6. SYSTEM OVERVIEW

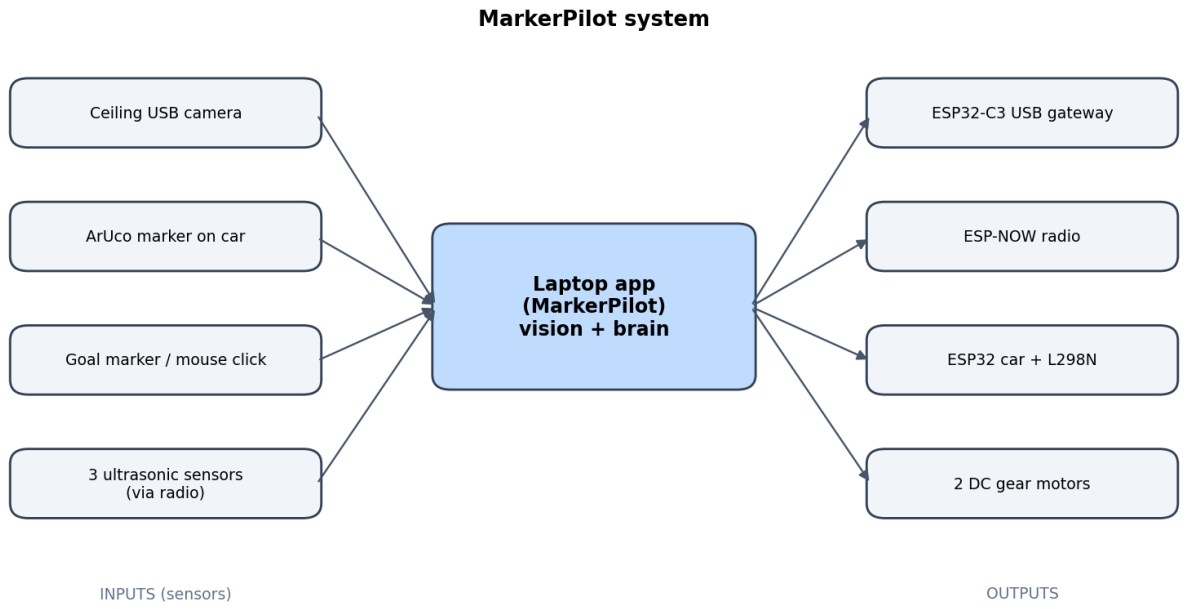


Figure 1: Block diagram

7. HOW I DID THIS (METHOD)

Step 1 - Seeing the car

OpenCV's ArUco detector finds the four corners of each marker. The centre gives the position and the direction from the centre to the middle of the top edge gives the heading. The known printed size (5 cm) tells the program how many pixels make one centimetre, so distances are shown in real centimetres.

Step 2 - Cleaning the measurements (Kalman filter)

An Extended Kalman Filter models the car as a unicycle with position, heading, speed and turn rate. It removes jitter, estimates speed, keeps tracking for a moment if the marker is hidden by glare, rejects impossible jumps, and predicts 0.08 s ahead to cancel the delay of the camera and radio.

Step 3 - Driving (two brains)

The classic controller turns on the spot when the target is more than 55° to the side, otherwise it drives forward while steering with a PD controller and brakes smoothly in the last 30 cm. The RL brain is a Q-learning agent: it looks at the distance band, the heading-error sector (16 sectors) and the obstacle situation, and picks one of 9 moves such as forward, creep-left or spin-right. It gets a reward for every centimetre of progress and for aiming at the goal, a big bonus for arriving and penalties for crashing or wasting time.

Step 4 - Practising in a simulator

I wrote a physics simulator of the car with motor dead-band, motor lag, unequal left/right motors, radio delay and ultrasonic sensors. It even draws a fake camera picture with real ArUco markers, so the complete vision pipeline is tested. Every training episode uses different random motors, delays, start points, goals and obstacles, so the agent cannot just memorise one situation (domain randomisation).

Step 5 - Safety and self-correction

A safety layer sits on top of both brains: it slows down and turns away from obstacles, drives a short detour so the car does not dither in front of a box, and backs out if the camera sees the car pushing without moving. Echoes farther away than the goal are ignored so the car can still reach a goal next to a wall. The app also compares what it commanded with what the camera saw: if the car drives backwards or turns the wrong way, it fixes the motor direction setting by itself.

Step 6 - A reliable radio link (firmware v2)

The car and the gateway pair by themselves: packets carry a 'M2' header, and the car answers the first MarkerPilot gateway it hears, so no MAC addresses are typed in. The car's three ultrasonic sensors are read with interrupts, so the program never waits for an echo and reacts to a new command within a millisecond or two. Motor power is ramped (0 to full in about 85 ms) because sudden reversals at full power pulled the battery voltage down and reset the ESP32. If no command arrives for 400 ms the motors ramp to a stop. As a last line of defence the car itself cuts forward motion when the front sensor sees an obstacle closer than the guard distance set by the app; turning and reversing still work so the app can steer away. Every telemetry packet reports the distances, signal strength, the motor power actually applied and which command it is answering, so the gateway can measure the delay. The command 'markerpilot linktest' prints the signal strength, packet loss, delay and sonar health, and can pulse each wheel to check the wiring.

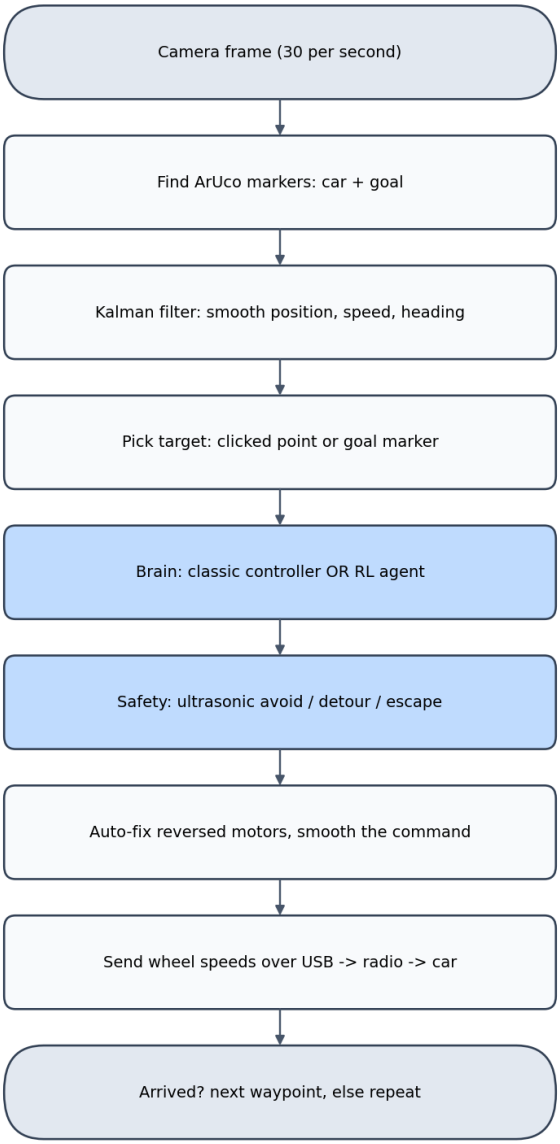


Figure 2: What happens in every camera frame

8. TESTING AND RESULTS

All numbers below come from running the real MarkerPilot code in the simulator with randomised car parameters (300 episodes each). The end-to-end test uses the rendered camera picture, so the ArUco detection and the Kalman filter are part of the test.

Test	Classic controller	RL agent
Goals reached - open floor	100%	100%
Average time - open floor	3.3 s	4.0 s
Goals reached - with obstacles	91%	94%
Average time - with obstacles	5.3 s	5.9 s

Table 3: Simulator results (300 randomised episodes per cell)

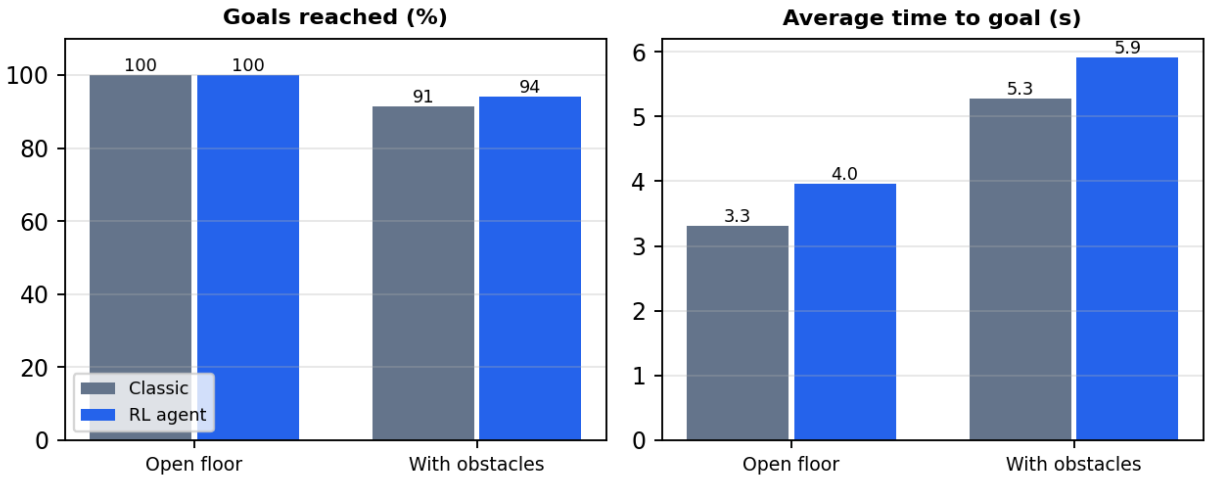


Figure 3: Classic controller vs RL agent

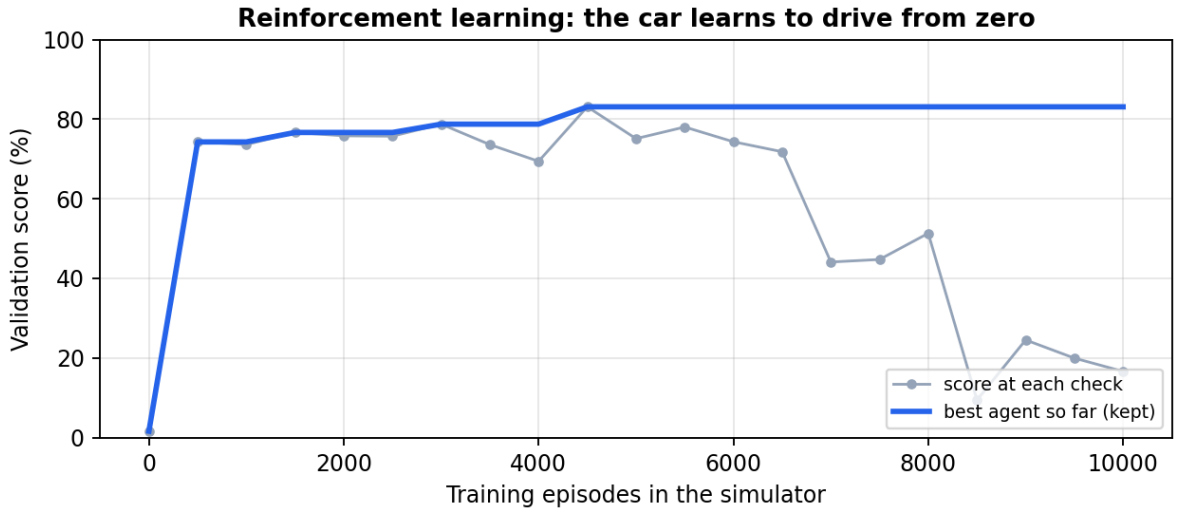


Figure 4: An RL agent starting from an empty table learns to reach about 80% of goals within a few thousand practice runs. The agent shipped with the app starts from simple driving rules and is improved further by training.

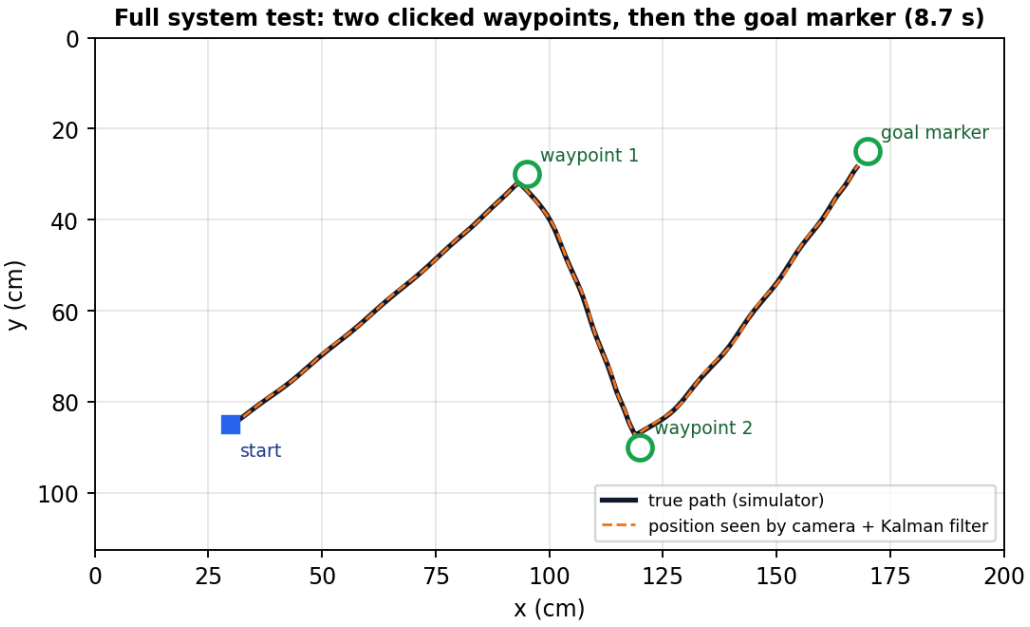


Figure 5: Full-system test in the simulator: the car visits two clicked waypoints and then the goal marker in 8.7 s. The dashed line is what the camera and Kalman filter believed; it lies on top of the true path.

Observation: the classic controller is slightly faster; the RL agent is just as reliable on an open floor and learned its behaviour from experience instead of hand-tuned rules. Both need the safety layer to get around obstacles, because the ultrasonic sensors only see what is in front of the car.

9. APPLICATIONS

- Warehouse trolleys that follow floor markers to shelves.
- Hospital and hotel delivery robots on a fixed floor area.
- Classroom kit for teaching computer vision, control and machine learning.
- Factory floors where one ceiling camera guides several small robots.

10. ADVANTAGES AND LIMITATIONS

- No position drift: the ceiling camera always knows where the car is.
- Cheap robot: no encoders, LiDAR or on-board camera needed.
- Same code runs in the simulator and on the real car.
- Safe by design: the car stops on its own if the radio goes quiet or an obstacle is too close.
- Limitation: works only inside the camera's view, and needs reasonable lighting.
- Limitation: obstacles are seen only by the ultrasonic sensors, not by the camera.

11. FUTURE IMPROVEMENTS

- Detect obstacles in the camera picture and plan a full path around them (A* search).
- Use several ceiling cameras for a bigger area.
- Control several cars at once, each with its own marker ID.
- Replace the Q-table with a small neural network (deep RL) for smoother driving.

12. CONCLUSION

MarkerPilot shows that a single ceiling camera, a printed marker and an ESP32 are enough for accurate autonomous navigation. The Kalman filter makes the position reliable, the classic controller and the RL agent both drive the car to its goals, and the simulator made it possible to test and train everything safely before using the real car.

13. REFERENCES

- OpenCV documentation - Detection of ArUco markers (docs.opencv.org).
- Sutton and Barto - Reinforcement Learning: An Introduction (Q-learning).
- Thrun, Burgard and Fox - Probabilistic Robotics (Extended Kalman Filter).
- Espressif - ESP-NOW protocol documentation (docs.espressif.com).